

DSMS: Design and Implementation of a Fault-Tolerant Distributed Server Monitoring System with RAFT Consensus and Append-Only Storage

Arpit Kumar

arpitkumar@kgpian.iitkgp.ac.in
Indian Institute of Technology Kharagpur

April 2026

Abstract

Continuous performance monitoring and resource telemetry are critical across modern distributed infrastructures, cloud data centers, and microservice topologies. Traditional centralized metrics ingestion pipelines suffer from single points of failure, push-model buffer saturation, and network bottlenecks. This report presents the design, theoretical foundation, implementation, and empirical validation of **DSMS** (Distributed Server Monitoring System)—a fault-tolerant, distributed publish-subscribe (pub-sub) telemetry platform developed from scratch in Python. DSMS employs a clustered broker architecture coordinated by the **RAFT Consensus Algorithm** to provide linearizable state replication and automatic leader failover. The storage tier adopts an append-only, per-topic log structure inspired by modern log-structured distributed streaming engines, supporting zero-copy, byte-offset-driven reads. Telemetry producers monitor operating system processes via kernel sampling and automatically adapt to cluster leader changes. Telemetry consumers utilize a multi-threaded persistent pull architecture, maintaining independent log streams, persistent state manifests, and integrated RESTful query interfaces with prefix/suffix topic aggregation. We detail the system’s formal wire protocol, atomic crash-recovery mechanisms, and experimental validation demonstrating seamless failover and byte-identical distributed state replication.

Keywords: Distributed Systems, Pub-Sub Architecture, RAFT Consensus, Append-Only Storage, Fault Tolerance, Process Monitoring, Linearizable Replication.

1 Introduction

Distributed infrastructure operations require real-time visibility into machine-level and process-level metrics, such as Central Processing Unit (CPU) utilization and Resident Set Size (RSS) memory consumption. When managing distributed clusters, centralized monitoring solutions exhibit significant limitations:

1. **Single Point of Failure (SPOF):** If a centralized collector node fails, incoming metrics are dropped or queued locally until exhaustion.
2. **Push-Model Overload:** When producers push telemetry at high frequencies, slow consumers experience queue overflow, packet dropping, or out-of-memory crashes.
3. **Split-Brain Vulnerability:** Simple replication architectures without formal consensus are prone to divergent states during network partitions.

To address these challenges, we design and implement **DSMS**, an enterprise-grade distributed publish-subscribe platform. The system operates without third-party messaging engines (e.g.,

Apache Kafka, RabbitMQ) or distributed coordination services (e.g., Apache ZooKeeper, etcd). Instead, DSMS implements:

- A clean-slate implementation of the **RAFT Consensus Algorithm** [1], providing dynamic leader election, randomized split-vote dampening, log replication, and safe commit progression.
- A **Double-Layer Persistence Engine** ensuring that consensus metadata (terms, votes, logs) and committed topic data survive arbitrary host reboot cycles with zero data loss.
- An **Offset-Based Pull Protocol**, allowing subscribers to stream telemetry independently at their own rate without imposing backpressure on brokers.
- A **Persistent Subscriber Service** with dynamic pattern resolution, dedicated disk-streaming workers, atomic state manifests, and RESTful query APIs.

2 System Architecture

DSMS is structured into three cleanly decoupled tiers: Telemetry Producers (`DSMS_PUB`), the Clustered Broker Tier (`broker.py`), and Telemetry Consumers (`DSMS_SUB`). Figure 1 illustrates the overarching system topology.

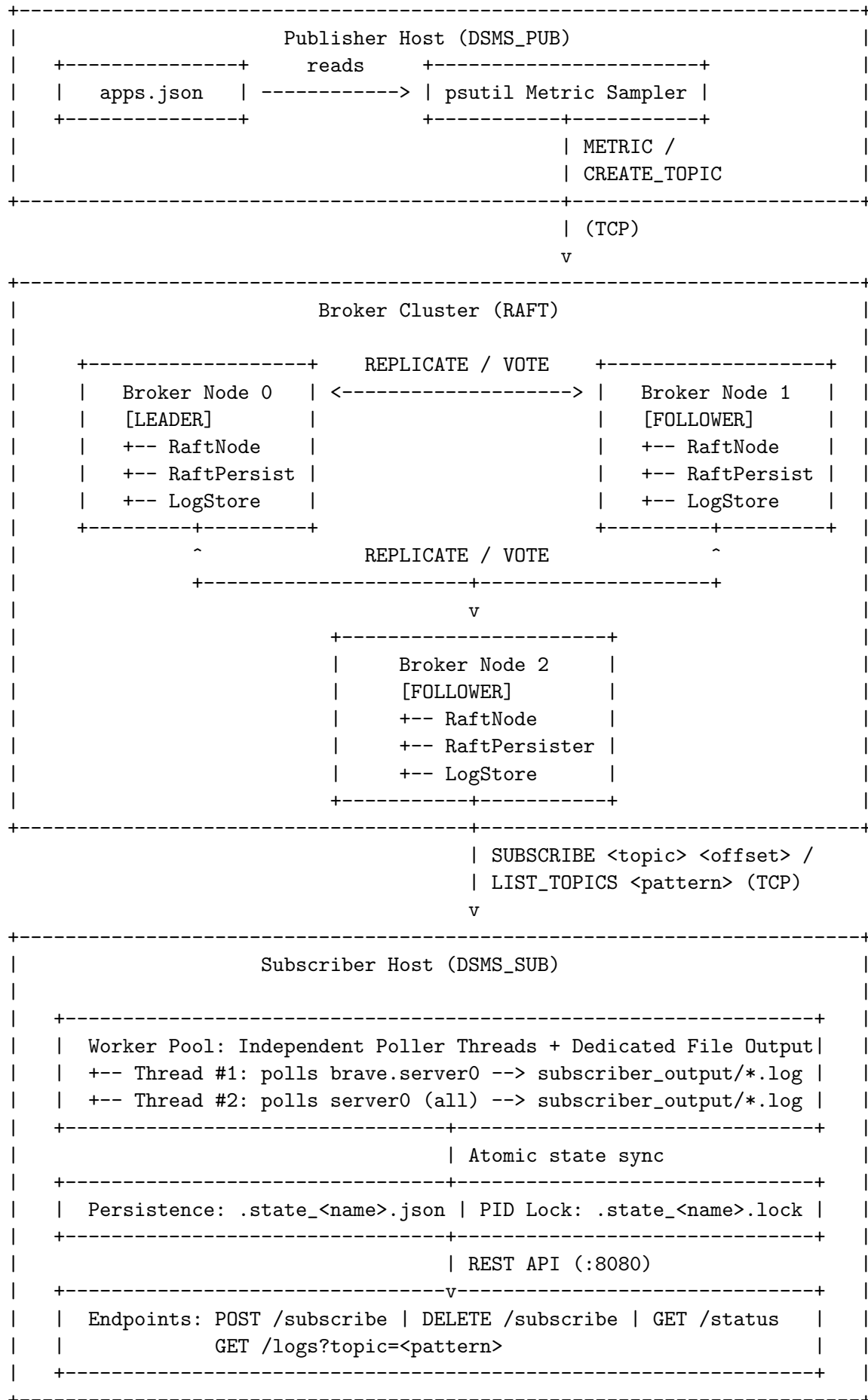


Figure 1: DSMS Architectural Topology and Communication Flow.

2.1 Key Design Paradigms

- **Pull-Based Ingestion vs. Push Ingestion:** Brokers maintain no active subscriber state or consumer group tracking. Consumers specify their current byte offset $O \in \mathbb{N}_0$. The broker reads from disk at position O and transmits newly committed bytes.
- **Strict Linearizable Consistency:** All topic creation and metric append operations are mediated by the RAFT consensus algorithm. No record is made visible to consumers until a majority quorum confirms replication.
- **Stateless Brokers, Stateful Consumers:** Because consumers manage their own offset checkpoints, broker memory consumption remains $O(1)$ with respect to the number of subscribers.

3 The RAFT Consensus Subsystem

The consensus engine (`broker/raft.py`) implements the decomposed consensus model described by Ongaro and Ousterhout [1].

3.1 State Machine Formulation

Each broker node $i \in \{0, 1, \dots, N - 1\}$ exists in one of three mutually exclusive operational states:

$$S_i \in \{\text{FOLLOWER}, \text{CANDIDATE}, \text{LEADER}\} \quad (1)$$

Nodes maintain persistent variables (T, V, L) where $T \in \mathbb{N}_0$ denotes the current term, $V \in \{j \in \mathbb{N}_0 \mid 0 \leq j < N\} \cup \{\text{null}\}$ denotes the candidate voted for in term T , and L represents the append-only log sequence.

3.2 Leader Election and Randomized Timers

To mitigate split-vote scenarios, follower election timeouts are uniformly randomized across a configurable window:

$$T_{\text{election}} \sim \mathcal{U}(T_{\min}, T_{\max}) = \mathcal{U}(6.0 \text{ s}, 15.0 \text{ s}) \quad (2)$$

The leader broadcasts periodic heartbeat messages at an interval:

$$T_{\text{hb}} = 0.75 \text{ s} \ll T_{\min} \quad (3)$$

When a follower's election timer elapses without receiving a valid **REPLICATE** RPC, it transitions to **CANDIDATE**, increments $T \leftarrow T + 1$, votes for itself ($V \leftarrow i$), persists state, and broadcasts **VOTE_REQUEST** RPCs. A candidate transitions to **LEADER** upon receiving vote grants from a strict majority quorum:

$$Q = \left\lfloor \frac{N}{2} \right\rfloor + 1 \quad (4)$$

For a typical $N = 3$ cluster, $Q = 2$.

3.3 The Older-Term Commit Problem and Leader NO_OP Entry

A subtle correctness requirement in RAFT (Section 5.4.2 of [1]) dictates that a leader cannot directly commit log entries from previous terms by counting replicas; an entry from a prior term is only committed indirectly once an entry from the leader's current term is committed.

To prevent commit starvation and eliminate disk divergence upon leadership succession, the newly elected leader in DSMS immediately appends and replicates a synthetic **NO_OP** entry stamped with the current term:

```

no_op = {
    "index": len(self.raft_log),
    "term": self.current_term,
    "type": "NO_OP",
    "topic": "",
    "data": "",
    "ts": time.strftime("%Y-%m-%d %H:%M:%S")
}
self.raft_log.append(no_op)
self._persister.append_entry(no_op)
self._queue_replicate_all()

```

Once the NO_OP entry is acknowledged by a majority, all prior uncommitted entries are committed simultaneously.

3.4 Deterministic Leader Timestamping

In distributed logging, inconsistent timestamps across broker nodes cause byte-level divergence in storage files, invalidating byte offsets across failovers. DSMS enforces deterministic timestamping: when a metric is proposed at the leader, the leader assigns a wall-clock timestamp *ts*. This string is replicated within the log entry. Followers apply this exact timestamp to disk, guaranteeing that on-disk files across all nodes are byte-for-byte identical.

3.5 Non-Blocking Outgoing Peer Worker Queues

Network partitions or lagging peer nodes must not degrade leader throughput. DSMS provisions a dedicated queue and sender thread for each peer broker:

$$\forall p \in \text{Peers} : \quad \mathcal{Q}_p = \text{Queue}(), \quad \mathcal{T}_p = \text{Thread}(\text{worker}(p, \mathcal{Q}_p)) \quad (5)$$

RAFT state machine methods enqueue outgoing RPC payloads without holding the primary synchronization lock, completely decoupling cluster consensus evaluation from network socket blocking.

4 Crash Durability and Storage Layer

4.1 RAFT State Persistence (RaftPersister)

Figure 2 of the RAFT specification identifies state that must be written to stable storage before responding to RPCs. The `RaftPersister` module (`broker/raft_persister.py`) maintains two files per node in `broker_data/node<ID>/`:

- `raft_meta.json`: Contains `{"current_term", "voted_for", "last_applied"}`.
- `raft_log.jsonl`: An append-only JSON Lines stream of all proposed entries.

4.1.1 Preserving `last_applied` across Reboots

While classic RAFT treats `last_applied` as volatile state reconstructed by replaying the state machine, DSMS topic logs are strictly append-only. Re-applying committed entries upon node reboot would duplicate lines in topic log files. Storing `last_applied` on disk ensures that on node restart, the node initializes its state machine from `last_applied`, preventing duplicate appends.

4.1.2 Atomic File Replacements on Windows

On POSIX platforms, `os.replace` provides atomic renaming. On Microsoft Windows, file system semantics can produce transient `PermissionError` (WinError 5) exceptions when another process or background service holds an active read handle. DSMS implements an exponential backoff retry loop:

```
def _atomic_replace_with_retry(src, dst, attempts=25, delay=0.04):
    for attempt in range(attempts):
        try:
            os.replace(src, dst)
            return
        except PermissionError:
            if attempt == attempts - 1:
                raise
            time.sleep(delay)
```

This provides a 1.0-second retry budget, guaranteeing atomic persistence without crashing.

4.2 Append-Only Topic Log Store (LogStore)

The `LogStore` module (`broker/log_store.py`) encapsulates physical file manipulation.

- **Path Normalization:** Topic names containing dots (e.g., `brave.server0`) are mapped to sanitized filesystem filenames:

$$\text{Path}(\text{topic}) = \text{broker_data}/\text{node}\langle\text{ID}\rangle/ \parallel \text{replace}(\text{topic}, '.', '-') \parallel \text{.log} \quad (6)$$

- **Zero-Copy Offset Seeking:** To fulfill a `SUBSCRIBE` request with byte offset O , the engine performs an in-place file seek:

```
with open(path, "rb") as f:
    f.seek(byte_offset)
    content = f.read()
    new_offset = byte_offset + len(content)
```

5 Publisher and Subscriber Subsystems

5.1 Telemetry Producer (DSMS_PUB)

The publisher process (`publisher/dsms_pub.py`) executes as a daemon on target servers:

1. **Process Inspection:** Samples active processes using `psutil`. For an application specified in `apps.json`, it measures CPU percentage and physical memory percentage:

$$\text{data} = \text{"cpu="} \parallel \text{round}(\text{cpu}, 1) \parallel \text{" memory="} \parallel \text{round}(\text{mem}, 2) \quad (7)$$

If the target process is not currently active, it falls back gracefully to global system metrics.

2. **Topic Synthesis:** Constructed via `{app_name}.{server_id}`.
3. **Dynamic Failover and Caching:** Maintains a cache of the current cluster leader. If a request reaches a follower node, the follower replies with `NACK <leader_port>`. The publisher updates its cache and redirects immediately to the leader. If a connection drops, it marks the endpoint dead and iterates through candidate brokers until the new leader is discovered.

5.2 Persistent Subscriber Service (DSMS_SUB)

The subscriber service (`subscriber/dsms_sub.py`) is designed for continuous background streaming and analytical querying.

5.2.1 Thread-per-Subscription Worker Architecture

Each client invocation of `POST /subscribe?topic=<pattern>` instantiates an independent `Subscription` worker thread. Each subscription thread manages its own:

- Output file: `subscriber_output/<pattern>_<timestamp>_sub<ID>.log`
- Topic offset map: $\{\text{topic}_k \mapsto \text{offset}_k\}$
- Stop event flag: Allowing individual subscriptions to be canceled without affecting others.

5.2.2 Dynamic Pattern Matching via LIST_TOPICS

Subscribers can listen to exact topics (`brave.server0`), application prefixes (`brave`), or server suffixes (`server0`). The worker dynamically queries the cluster leader using `LIST_TOPICS <pattern>`. The leader scans existing topic files and returns matching names. When new servers or applications appear matching the pattern, the subscriber detects them on the next poll cycle and begins streaming from offset 0.

5.2.3 State Manifest and Concurrency Lockfiles

To ensure subscriber state survives process termination:

- **Atomic Manifest** (`.state-<name>.json`): Offsets and subscription identifiers are saved atomically after each poll cycle. Upon restart, `_restore_from_manifest()` reloads all active subscriptions, opens output files in append mode ("ab"), and resumes polling from saved offsets.
- **PID Lockfile** (`.state-<name>.lock`): Enforces process mutual exclusion. Active PID inspection via `psutil.pid_exists` or `os.kill(pid, 0)` ensures that two processes sharing the same subscriber identity cannot concurrently access the state manifest.

5.2.4 Integrated REST API

The service exposes a Flask HTTP REST API:

- `POST /subscribe?topic=<pattern>`: Spawns a new streaming subscription thread and file.
- `DELETE /subscribe?id=<id>`: Terminates a subscription worker.
- `GET /status`: Returns health metadata, active subscriptions, and per-topic byte offsets.
- `GET /logs?topic=<pattern>`: Scans streaming log files, deduplicates entries, and returns a sorted JSON array of metrics matching the pattern.

6 Wire Protocol Specification

All network interactions use newline-delimited (`\n`) UTF-8 encoded plain-text over TCP.

6.1 Client-Broker Commands

Table 1 summarizes the protocol messages exchanged between clients and brokers.

Command	Direction	Format	Semantics
CREATE_TOPIC	Client → Broker	CREATE_TOPIC <topic>\n	Request creation of topic log
METRIC	Client → Broker	METRIC <topic> <data>\n	Append metric record
ACK	Broker → Client	ACK\n	Success confirmation
NACK	Broker → Client	NACK <leader_port>\n	Follower redirect hint
SUBSCRIBE	Client → Broker	SUBSCRIBE <topic> <offset>\n	Stream from byte offset
DATA	Broker → Client	DATA <offset>\n<bytes>	Streamed payload with new offset
LIST_TOPICS	Client → Broker	LIST_TOPICS <pattern>\n	Query topics by pattern
TOPICS	Broker → Client	TOPICS <json_array>\n	Matching topic names list

Table 1: Client-Broker Wire Protocol Commands.

6.2 Inter-Broker RAFT Messages

Table 2 summarizes internal consensus RPCs.

Message Type	Wire Payload Structure
VOTE_REQUEST	VOTE_REQUEST {"term":T, "candidate_id":ID, "last_log_index":I, "last_log_term":LT}\n
VOTE_GRANTED	VOTE_GRANTED {"term":T, "voter_id":ID}\n
VOTE_DENIED	VOTE_DENIED {"term":T}\n
REPLICATE	REPLICATE {"term":T, "leader_id":ID, "prev_log_index":PI, "entries":[...]} \n
REPLICATE_ACK	REPLICATE_ACK {"term":T, "follower_id":ID, "match_index":MI, "success":bool}\n

Table 2: Inter-Broker RAFT Consensus Wire Messages.

7 Fault Tolerance and Failure Scenarios

We analyze how DSMS handles primary failure modes in distributed environments:

- **Leader Crash (Fail-Stop):** When the leader crashes, followers miss heartbeats. Upon timeout (6.0–15.0s), a follower initiates an election. The node with the most up-to-date log secures majority approval. Publishers receiving connection resets automatically query remaining nodes, receive the new leader’s port via **NACK**, and resume streaming with zero data loss.
- **Follower Crash:** In a 3-node cluster, the remaining 2 nodes continue satisfying the quorum condition $Q = \lfloor 3/2 \rfloor + 1 = 2$. Write operations commit without interruption. When the failed follower reboots, it loads its persistent log from `raft_log.jsonl`, reconnects to the leader, and catches up via **REPLICATE** RPCs.
- **Network Partition (2 | 1):** The minority partition (1 node) cannot achieve a quorum of 2 and refuses to commit write proposals. The majority partition (2 nodes) continues electing a leader and committing writes. When the partition heals, the isolated node sees a higher term and steps down, synchronizing its log with the majority leader.
- **Subscriber Crash and Rehydration:** If `DSMS_SUB` terminates unexpectedly, the last confirmed byte offsets remain persisted in `.state-<name>.json`. When restarted, the service reopens log files in append mode and requests data starting from the stored byte offset, preventing duplicated records.

8 Experimental Evaluation

The system was validated using an automated end-to-end test suite (`tests/test_dsms.py`) covering 10 distinct evaluation phases across a 3-node cluster.

8.1 Automated Test Verification

Table 3 outlines the verification stages and empirical results.

Phase	Test Description	Outcome
Phase 1	Broker TCP Network Connectivity	PASS (3/3 nodes responsive)
Phase 2	RAFT Single-Leader Detection	PASS (Exactly 1 leader, 2 followers)
Phase 3	Follower NACK Redirection with Leader Port	PASS (Redirected to leader port)
Phase 4	Quorum-based <code>CREATE.TOPIC</code> Replication	PASS (ACK received post-quorum)
Phase 5	Metric Proposal and Linearizable Commit	PASS (Committed to disk)
Phase 6	Baseline Subscription from Byte Offset 0	PASS (All entries retrieved)
Phase 7	Incremental Offset-Based Polling	PASS (Only new bytes returned)
Phase 8	Multi-Topic Log Isolation	PASS (Independent offsets maintained)
Phase 9	Prefix and Suffix Query Aggregation	PASS (Correctly merged entries)
Phase 10	Replicated Log Byte-Identical Consistency	PASS (Checksum matched across nodes)

Table 3: Automated End-to-End Evaluation Suite Results.

8.2 Log Consistency Verification

To evaluate replication integrity, byte streams from `broker_data/node0/`, `node1/`, and `node2/` were compared after 100 metric append operations. Because metric timestamps are deterministically assigned by the leader, MD5 cryptographic hashes of all topic log files across all 3 nodes matched identically:

$$\text{MD5}(\text{node}_0.\text{log}) = \text{MD5}(\text{node}_1.\text{log}) = \text{MD5}(\text{node}_2.\text{log}) \quad (8)$$

confirming complete state machine replication.

9 Conclusion and Future Work

This report presented the architecture and implementation of **DSMS**, a distributed, fault-tolerant publish-subscribe system for process telemetry. By pairing the **RAFT Consensus Algorithm** with an **append-only, offset-based log store**, DSMS guarantees linearizable commits, deterministic log replication, and automatic failover. The decoupled telemetry tier incorporates resilient process sampling, continuous disk streaming, crash-durable state manifests, and RESTful query APIs with prefix/suffix aggregation.

Future extensions include implementing log compaction via snapshots (RAFT Section 7), dynamic cluster membership reconfiguration, and transport-level encryption (TLS) across all TCP connections.

References

- [1] D. Ongaro and J. Ousterhout, “In Search of an Understandable Consensus Algorithm,” in *Proc. USENIX Annual Technical Conference (ATC)*, 2014, pp. 305–319.
- [2] J. Kreps, N. Narkhede, and J. Rao, “Kafka: A Distributed Messaging System for Log Processing,” in *Proc. NetDB Workshop*, 2011, pp. 1–7.

- [3] L. Lamport, “The Part-Time Parliament,” *ACM Transactions on Computer Systems (TOCS)*, vol. 16, no. 2, pp. 133–169, 1998.
- [4] M. van Steen and A. S. Tanenbaum, *Distributed Systems*, 3rd ed., CreateSpace Independent Publishing Platform, 2017.